

DIY Kiosk for Raspberry Pi OS

title: DIY Kiosk for Raspberry Pi OS

updated: 2026-06-16 23:21:50Z

created: 2026-06-16 19:40:55Z

latitude: -22.56088070

longitude: 17.06575490

altitude: 0.0000

- [1. Introduction](#)
- [2. Requirements](#)
- [3. Architecture Overview](#)
 - [3.1 Component Selection](#)
 - [3.2 Boot and Login flow](#)
- [4. Setup & Configuration](#)
 - [4.1 Package Installation](#)
 - [4.1.1 Base packages + greetd & labwc](#)
 - [4.1.2 GTK Application](#)
 - [4.1.3 FreeRDP](#)
 - [4.2 User Account](#)
 - [4.2.1 Create Dedicated Kiosk User](#)
 - [4.3 Custom Application Components](#)
 - [4.3.1 Overview](#)
 - [4.3.2 kiosk-session](#)
 - [4.3.3 kiosk-controller](#)
 - [4.3.4 kiosk-login.py](#)
 - [4.3.5 kiosk-rdp](#)
 - [4.3.6 kiosk-logout](#)
 - [4.4 systemd User Services](#)
 - [4.4.1 Overview](#)
 - [4.4.2 kiosk-controller.service](#)
 - [4.4.3 kiosk-login.service](#)
 - [4.4.4 kiosk-rdp.service](#)
 - [4.5 Display Management](#)

- [4.5.1 Enable seatd](#)
- [4.5.2 Configuring greetd](#)
- [4.5.3 Configuring labwc](#)
- [4.6 Power Management](#)
 - [4.6.1 Enable EEPROM](#)
 - [4.6.2 Schedule the suspend + wake](#)
 - [4.6.3 Sync time with NTP](#)
- [4.7 GTK Login Application](#)
 - [4.7.1 Application Overview](#)
 - [4.8.2 User Interface](#)
 - [4.8.3 Credential Handling](#)
- [4.8 Session Manager](#)
 - [4.8.1 Connection Workflow](#)
 - [4.8.2 Connection Parameters](#)
- [4.9 Security Hardening](#)
 - [4.9.1 Remove Desktop Escape Route](#)
 - [4.9.2 Restrict Kiosk User](#)
 - [4.9.3 Protect Credentials](#)

1. Introduction

We want to migrate from Clearcube small factor PCs to Raspberry Pis.

At the backend instead of using Inuvika's infrastructure server, we would like to replace that layer with Windows RDS so that the Pis can use RDP to connect to a session/client/profile served/stored by RDS.

2. Requirements

We already examined options like No Touch OS and WTware and they do not fit our needs as No Touch OS licensing is too expensive and the WTware is outdated especially in UI/UX, documentation and it's core community.

So we would like to go with a model like:

Raspberry Pi OS + FreeRDP + DIY Kiosk

Then each Pis would reach out to the RDS for a windows session via RDP.

We plan to use the raspberry Pis in the library for hundreds of students to access.

Since we are planning to deploy at the organization scale, we want to use tools/components within this DIY solution that:

- is actively maintained
- alot of documentation
- scales well with deployment and ease of management

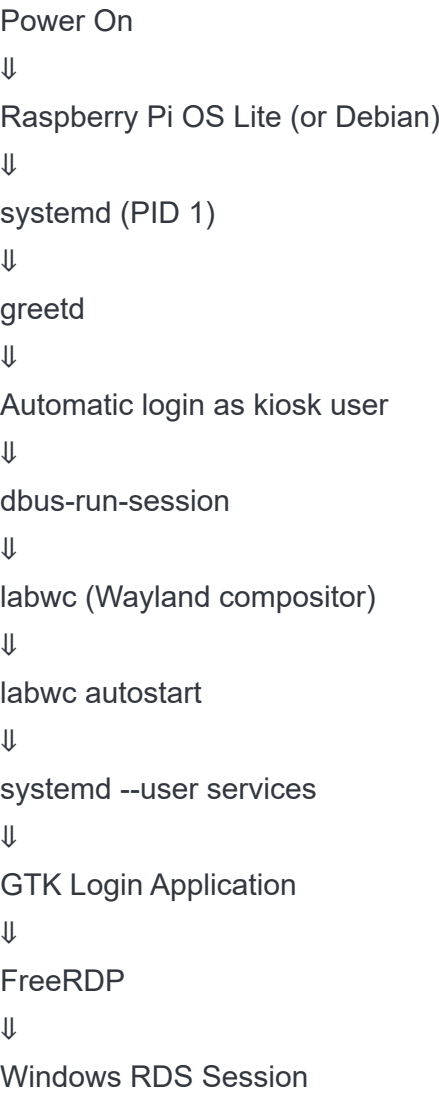
3. Architecture Overview

3.1 Component Selection

Name	Purpose
Raspberry Pi OS Lite	Minimal, headless Debian-based OS (no desktop environment) that serves as the lightweight foundation for the kiosk.
greeterd	Lightweight display/login manager that starts the graphical session and performs the automatic login as the dedicated kiosk user.
labwc	Minimal Wayland compositor that provides the graphical environment in which the login UI and FreeRDP run.
GTK	GUI toolkit used to build the simple Python login window (username/password entry) that students see on boot.

Name	Purpose
FreeRDP	

3.2 Boot and Login flow



4. Setup & Configuration

Flash Raspberry Pi OS Lite

Initial Configuration:

- Hostname
- SSH
- Locale
- Timezone

4.1 Package Installation

4.1.1 Base packages + greetd & labwc

```
sudo apt update && sudo apt upgrade -y
```

```
sudo apt install \  
  greetd \  
  labwc \  
  seatd \  
  util-linux-extras \  
  dbus-user-session \  
  policykit-1 \  
  xdg-desktop-portal \  
  xdg-desktop-portal-wlr
```

4.1.2 GTK Application

```
sudo apt install -y python3 python3-gi gir1.2-gtk-4.0
```

4.1.3 FreeRDP

```
sudo apt install freerdp3-wayland
```

4.2 User Account

Keep your administrative account.

4.2.1 Create Dedicated Kiosk User

Create:

```
sudo adduser kiosk
```

Add required groups:

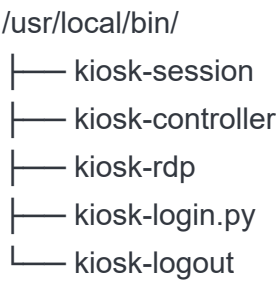
```
sudo usermod -aG video,input,audio,render,seat kiosk
```

Verify:

```
id kiosk
```

4.3 Custom Application Components

4.3.1 Overview



Script	Purpose
kiosk-session	Create labwc session
kiosk-controller	Orchestrates login and RDP workflow
kiosk-rdp	Launches FreeRDP using supplied credentials
kiosk-login.py	GTK login application
kiosk-logout	Session cleanup (future)

4.3.2 kiosk-session

Create: `sudo vi /usr/local/bin/kiosk-session`

File:

```
#!/bin/sh

export XDG_RUNTIME_DIR="/run/user/$(id -u)"
export XDG_SESSION_TYPE=wayland
export XDG_SESSION_DESKTOP=labwc
export XDG_CURRENT_DESKTOP=labwc

exec dbus-run-session labwc
```

Make it executable: `chmod +x /usr/local/bin/kiosk-session`

4.3.3 kiosk-controller

Create: `sudo vi /usr/local/bin/kiosk-controller`

File:

```
#!/bin/bash

CREDS="/run/user/$(id -u)/kiosk-creds"
```

```

while true
do
    rm -f "$CREDS"

    # 1. start login
    systemctl --user start kiosk-login.service

    # 2. wait until login exits
    while systemctl --user is-active kiosk-login.service >/dev/null
    do
        sleep 0.2
    done

    # 3. validate creds
    if [ ! -f "$CREDS" ]; then
        continue
    fi

    # 4. run RDP
    systemctl --user start kiosk-rdp.service

    # 5. wait until RDP exits
    while systemctl --user is-active kiosk-rdp.service >/dev/null
    do
        sleep 1
    done
done

```

Make it executable: `chmod +x /usr/local/bin/kiosk-session`

4.3.4 kiosk-login.py

Create: `sudo vi /usr/local/bin/kiosk-login.py`

File:

```

#!/usr/bin/env python3

import gi
import os
from pathlib import Path

gi.require_version("Gtk", "4.0")
from gi.repository import Gtk

```

```
class LoginWindow(Gtk.ApplicationWindow):
    def __init__(self, app):
        super().__init__(application=app)

        self.set_title("Library Login")
        self.set_default_size(400, 250)
        self.set_resizable(False)

        box = Gtk.Box(
            orientation=Gtk.Orientation.VERTICAL,
            spacing=10
        )

        box.set_margin_top(30)
        box.set_margin_bottom(30)
        box.set_margin_start(30)
        box.set_margin_end(30)

        self.username = Gtk.Entry()
        self.username.set_placeholder_text("Username")

        self.password = Gtk.Entry()
        self.password.set_placeholder_text("Password")
        self.password.set_visibility(False)

        button = Gtk.Button(label="Login")
        button.connect("clicked", self.on_login)

        box.append(self.username)
        box.append(self.password)
        box.append(button)

        self.set_child(box)

        # Put cursor in username field immediately
        self.username.grab_focus()

    def on_login(self, button):
        user = self.username.get_text().strip()
        pw = self.password.get_text()

        if not user or not pw:
```

```

        return

    uid = os.getuid()

    cred_file = Path(f"/run/user/{uid}/kiosk-creds")

    try:
        with open(cred_file, "w") as f:
            f.write(f"{user}:{pw}\n")

        os.chmod(cred_file, 0o600)

    except Exception as e:
        print(f"Failed to write credentials: {e}")
        return

    self.get_application().quit()

class App(Gtk.Application):
    def __init__(self):
        super().__init__(
            application_id="org.library.kiosk"
        )

    def do_activate(self):
        win = LoginWindow(self)
        win.present()

app = App()
app.run(None)

```

Make it executable: `chmod +x usr/local/bin/kiosk-login.py`

4.3.5 kiosk-rdp

Create: `sudo vi /usr/local/bin/kiosk-rdp`

File:

```
#!/bin/bash
```

```
CREDS="/run/user/${id -u}/kiosk-creds"
```

```
if [ ! -f "$CREDS" ]; then
    exit 1
fi

USER=$(cut -d: -f1 "$CREDS")
PASS=$(cut -d: -f2- "$CREDS")

exec /usr/bin/wlfreendp3 \
    /v:10.13.14.142 \
    /u:"$USER" \
    /p:"$PASS" \
    /f \
    /dynamic-resolution \
    /cert:ignore \
    +clipboard \
    /sound:sys:pulse \
    /network:auto
```

Make it executable: `chmod +x usr/local/bin/kiosk-login.py`

4.3.6 kiosk-logout

TODO

4.4 systemd User Services

4.4.1 Overview

4.4.2 kiosk-controller.service

Starts the controller

```
[Unit]
Description=Kiosk Session Controller
After=graphical-session.target

[Service]
Type=simple
ExecStart=/usr/local/bin/kiosk-controller
Restart=always

[Install]
WantedBy=default.target
```

4.4.3 kiosk-login.service

Starts the GTK login.

```
[Unit]
Description=GTK Login

[Service]
Type=simple
Environment=GSK_RENDERER=cairo
Environment=LIBGL_ALWAYS_SOFTWARE=1
ExecStart=/usr/bin/python3 /usr/local/bin/kiosk-login.py
Restart=no

[Install]
WantedBy=default.target
```

4.4.4 kiosk-rdp.service

Connects to the RDS

```
[Unit]
Description=RDP Session

[Service]
Type=simple
```

```
ExecStart=/usr/local/bin/kiosk-rdp
```

```
Restart=on-failure
```

```
[Install]
```

```
WantedBy=default.target
```

4.5 Display Management

4.5.1 Enable seatd

Wayland compositors require seat management.

Enable:

```
sudo systemctl enable seatd && sudo systemctl start seatd
```

4.5.2 Configuring greetd

Edit: `sudo vi /etc/greetd/config.toml`

Contents:

```
[terminal]
```

```
vt = 1
```

```
[default_session]
```

```
command = "dbus-run-session labwc"
```

```
user = "kiosk"
```

4.5.3 Configuring labwc

Create:

```
sudo -u kiosk mkdir -p /home/kiosk/.config/labwc
```

4.6 Power Management

Now let's add scheduled power off and on for the raspberry PI to save power.

A Raspberry Pi that is fully powered off cannot wake itself up by software alone. There is no mechanism that can turn the board back on when it is completely unpowered.

Thus, we need to place it in the lowest practical power state the Pi can enter while still being able to wake itself, this is called **Deep Standby** and to achieve this power state we need to configure the onboard **EEPROM** bootloader settings.

And with the correct **EEPROM settings** you can:

1. Set an RTC wake alarm.
2. Enter a very low-power halt state (~3 mA / ~0.01–0.02 W).
3. Automatically power back on when the alarm fires → full boot → greetd → your kiosk stack.

4.6.1 Enable EEPROM

i) Edit the EEPROM config to put the Pi in a low power state

```
sudo rpi-eeprom-config --edit
```

Set (or add) these lines:

```
POWER_OFF_ON_HALT=1
WAKE_ON_GPIO=0
```

4.6.2 Schedule the suspend + wake

i) Create a suspend + wake script at `/usr/local/bin/kiosk-shutdown`:

```
#!/bin/bash
# Reliable RTC wake + accurate time keeping

set -e

RTC="/sys/class/rtc/rtc0/wakealarm"

# === CONFIGURE YOUR TIMES HERE ===
WAKE_HOUR=7
WAKE_MINUTE=30
# =====

if [ "${id -u}" -ne 0 ]; then
    echo "Must run as root"
```

```

    exit 1
fi

echo "=== Syncing time from network ==="

# Force a time sync (works with systemd-timesyncd)
timedatectl set-ntp true
sleep 2

# Wait until the clock is actually synchronised (max 15 seconds)
for i in {1..15}; do
    if timedatectl status | grep -q "System clock synchronized: yes"; then
        echo "Time is synchronized"
        break
    fi
    sleep 1
done

# Write the accurate system time into the hardware RTC
hwclock --systohc
echo "Hardware RTC updated: $(hwclock -r)"

# Clear any previous alarm
echo 0 > "$RTC"

# Calculate relative seconds until next wake time
NOW=$(date +%s)
WAKE_TODAY=$(date -d "today ${WAKE_HOUR}:${WAKE_MINUTE}" +%s)

if [ "$WAKE_TODAY" -gt "$NOW" ]; then
    SECONDS_UNTIL_WAKE=$((WAKE_TODAY - NOW))
else
    WAKE_TOMORROW=$(date -d "tomorrow ${WAKE_HOUR}:${WAKE_MINUTE}" +%s)
    SECONDS_UNTIL_WAKE=$((WAKE_TOMORROW - NOW))
fi

# Set relative wakealarm (the reliable method)
echo "+${SECONDS_UNTIL_WAKE}" > "$RTC"

echo "Wakealarm set: +${SECONDS_UNTIL_WAKE} seconds"
echo "Will wake at approximately: $(date -d @"$(NOW + SECONDS_UNTIL_WAKE)"))"

logger "Kiosk shutdown: RTC synced and wake set for +${SECONDS_UNTIL_WAKE}s"

```

```
# Enter low-power state
/sbin/halt
```

ii) Create a **systemd** service + timer to execute the script as a set time.

Create `/etc/systemd/system/kiosk-shutdown.service`:

```
[Unit]
Description=Scheduled kiosk low-power shutdown

[Service]
Type=oneshot
ExecStart=/usr/local/bin/kiosk-scheduled-shutdown.sh
```

Create `/etc/systemd/system/kiosk-shutdown.timer`:

```
[Unit]
Description=Daily kiosk shutdown timer

[Timer]
OnCalendar=*-*-* 22:00:00
Persistent=true

[Install]
WantedBy=timers.target
```

iii) Enable the service

```
sudo systemctl daemon-reload
sudo systemctl enable --now kiosk-shutdown.timer
```

4.6.3 Sync time with NTP

This is important because the Pi needs to have the accurate time, as it is needed by **systemd timer** to suspend at the correct time.

i) Point **timesyncd** at an internal time server with the config.

Create the config file `/etc/systemd/timesyncd.conf`:

```
[Time]
NTP=172.28.253.253

RootDistanceMaxSec=30
```

ii) Configure **system-timesyncd** to use NTP with the following commands:

```
sudo systemctl restart systemd-timesyncd  
sudo timedatectl set-ntp true
```

iii) Verify the configuration

```
timedatectl  
timedatectl timesync-status
```

TODO:

4.7 GTK Login Application

4.7.1 Application Overview

4.8.2 User Interface

4.8.3 Credential Handling

4.8 Session Manager

4.8.1 Connection Workflow

4.8.2 Connection Parameters

4.9 Security Hardening

4.9.1 Remove Desktop Escape Route

4.9.2 Restrict Kiosk User

4.9.3 Protect Credentials